

Databases, SQL and your first queries

Build a small database in your own BigQuery project, then ask it questions.

Tarek Atwan · Lead Instructor

Sun 27 Sep · 4:00–8:00 PM

BY THE END OF TODAY, YOU CAN

Class objectives

- 01 State what one row is in each table
 - 02 Upload a CSV and check its schema
 - 03 Choose and rename columns; sort and limit rows
 - 04 Filter rows with WHERE and IS NULL
 - 05 Explain two queries that return wrong rows silently
-

Warm-up

Five questions on your phone, ungraded. The last one tells your TA where you are with BigQuery.

Suggested time • 10 minutes

PART 02

How we talk to a database

Where the data lives now, and the language you use to ask it questions.

Most business data lives in a database

Last week: data lives in files and systems. Today: where most of it ends up.

Suggested time • 10 minutes

DEFINITION

A database is a set of tables stored and managed by software that answers questions about them.

Three ways data gets into a database

01

An application writes it

Each sale at the till adds a row.

02

A pipeline loads it

Scheduled loads from other systems.

03

You upload a file

Every database allows it. Today.

SQL is how you ask

One query, read aloud as a sentence. The syntax comes later today.

Suggested time • 8 minutes

DEFINITION

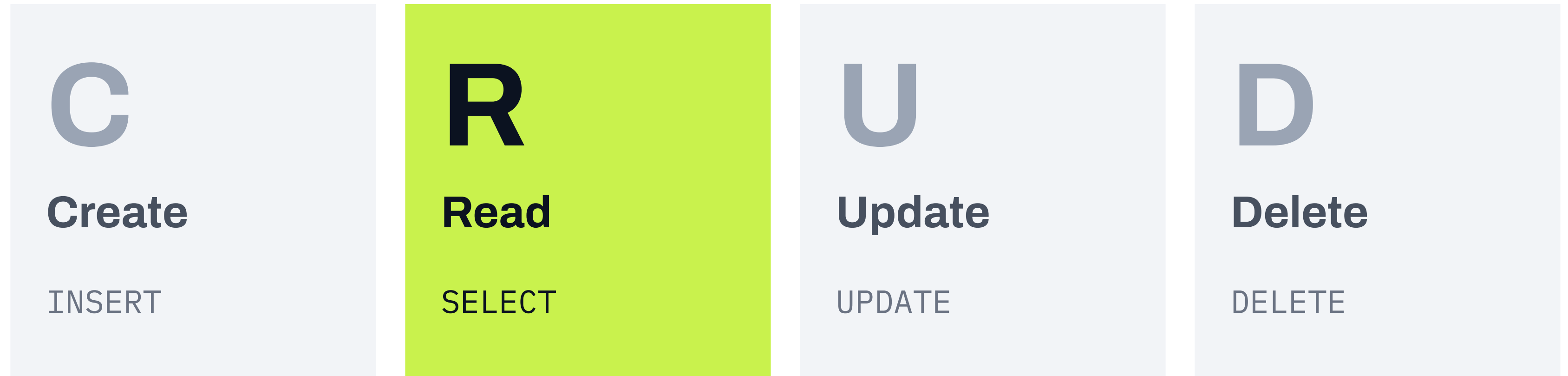
SQL, Structured Query Language, is the language of relational databases: you describe the result you want, and the database works out how to get it.

Read a query as a sentence

“Give me the order number and date of every completed order.”

```
SELECT order_id, order_date  
FROM D03.orders  
WHERE status = 'completed';
```

Four things you can do to data



The till creates rows; analysts read them. The BigQuery Sandbox allows reading only.

One row is one _____

For each table, type your answer in the chat before I show it.

Suggested time • 12 minutes

ONE ROW IS ONE ___ ?

order_items

The first 8 of its 12 rows.

Primary key

Foreign key

item_id	order_id	product_id	qty	price
9001	1001	P-77	2	19.50
9002	1001	P-14	1	8.00
9003	1002	P-77	3	19.50
9004	1002	P-05	1	42.00
9005	1003	P-14	2	8.00
9006	1003	P-23	1	26.00
9007	1003	P-77	1	19.50
9008	1004	P-05	2	42.00

Order 1001 has two rows. If one row were one order, 1001 would appear once.

ONE ROW IS ONE ___ ?

customers

Omar Nasser appears twice, as C-204 and C-401: two records, one person.

Primary key

customer_id	customer_name	city	segment	signup_date
C-118	Lina Haddad	Amman	Retail	2025-11-04
C-204	Omar Nasser	Irbid	retail	2026-01-22
C-377	Rana Khoury	Amman	RETAIL	2026-03-15
C-401	Omar Nasser	Irbid	Wholesale	2026-04-02
C-455	Sami Odeh	NULL	Wholesale	2026-05-19

DEFINITION

A primary key is the column whose value is different on every row, so it names exactly one row.

DEFINITION

A foreign key is a column that points to one row in another table; tables linked this way are what relational means.

Build your first database

The four D01 tables, uploaded into your own BigQuery project.

Open BigQuery and create the dataset

Download the four CSVs with Download raw file. Do not open them in Excel.

Suggested time • 10 minutes

[labs/D03/01-Evr-Create-Your-Dataset/README.md](#)

CONCEPT

Project, dataset, table

Project: your space in Google Cloud.

Dataset: a folder of tables in one region.

Table: the rows.

Create **D03** in Multi-region US, where the public datasets are.

PROJECT your-project-id

DATASET D03 • US

customers

orders

order_items

products

CONCEPT

Project, dataset, table

Project: your space in Google Cloud.

Dataset: a folder of tables in one region.

Table: the rows.

Create **D03** in Multi-region US, where the public datasets are.

PROJECT your-project-id

DATASET D03 • US

customers

orders

order_items

products

CONCEPT

Project, dataset, table

Project: your space in Google Cloud.

Dataset: a folder of tables in one region.

Table: the rows.

Create **D03** in Multi-region US, where the public datasets are.

PROJECT your-project-id

DATASET D03 · US

customers

orders

order_items

products

Upload orders together

Copy each field as I fill it in. Checkpoint: 5 rows in D03.orders.

Suggested time • 12 minutes

[labs/D03/01-Evr-Create-Your-Dataset/README.md](https://github.com/DataCamp/labs/blob/main/D03/01-Evr-Create-Your-Dataset/README.md)

Create a table from a file

- 1 Dataset : > Create table
- 2 Create table from: Upload
- 3 File: orders.csv · table: orders
- 4 Schema: Auto detect
- 5 Header rows to skip = 1**
- 6 Check Schema, Preview, Details

Upload products and meet `string_field_0`

The same steps as orders. Then open the Schema tab.

Suggested time • 10 minutes

products loaded without its header

What loaded

column	type
string_field_0	STRING
string_field_1	STRING
string_field_2	STRING

Every column is text, so the header row looks like data.

The fix

Delete it. Auto detect off, Edit as text on:

```
product_id:STRING,  
product_name:STRING,  
category:STRING
```

Header rows to skip = 1

Upload the last two and check

Checkpoint: four tables in Explorer with 5, 12, 5 and 5 rows.

Suggested time • 13 minutes

[labs/D03/01-Evr-Create-Your-Dataset/README.md](https://github.com/GoogleCloudPlatform/ai-examples/blob/main/labs/D03/01-Evr-Create-Your-Dataset/README.md)

DEFINITION

NULL is how a database says “no value”: an empty cell in the CSV loads as NULL, as C-455’s city does.

The first SELECT

Choose columns, sort, limit and filter. Predict the row count before every run.

SELECT and FROM

Before every run: one row is one ____, and how many rows will come back?

Suggested time • 8 minutes

[labs/D03/02-Ins-First-Select/README.md](#)

SELECT * means every column

SELECT chooses the columns. ***** means all of them.

```
SELECT *
```

```
FROM `your-project-id.D03.orders`;
```

One row is one order. How many rows?

FROM names the table

FROM says where the rows live: project, dataset, table, in backticks.

```
SELECT *  
FROM `your-project-id.D03.orders` ;
```

In your own project, FROM D03.orders is enough. 5 rows, 4 columns.

Choose columns, rename, sort, limit

Say the row count before you run. Checkpoint: item 9010 comes first.

Suggested time • 10 minutes

[labs/D03/03-Evr-Columns-Sort-Limit/README.md](https://github.com/DataCamp/labs/blob/main/D03/03-Evr-Columns-Sort-Limit/README.md)

ORDER BY sorts the result

DESC puts the largest qty first. **item_id** decides ties.

```
SELECT item_id, order_id, product_id, qty
FROM D03.order_items
ORDER BY qty DESC, item_id
LIMIT 3;
```

LIMIT keeps the first rows

LIMIT 3 keeps three rows of the sorted result.

```
SELECT item_id, order_id, product_id, qty  
FROM D03.order_items  
ORDER BY qty DESC, item_id  
LIMIT 3;
```

→ **9010, 9003, 9009**

Checkpoint: item 9010, qty 4, comes first.

WHERE, and two dead ends

One query BigQuery refuses to run, one that runs and gives the wrong answer.

Suggested time • 8 minutes

[labs/D03/04-Ins-Where-and-Null/README.md](#)

WHERE keeps the rows that pass

Text goes in single quotes. `<>` means “is not”.

```
SELECT order_id, customer_id, order_date, status
FROM D03.orders
WHERE status = 'completed';
```

→ 4 rows

Which customers have no city?

Predict 1 row. BigQuery refuses to run it.

```
SELECT customer_id, customer_name, city  
FROM D03.customers  
WHERE city = NULL;
```

→ Error: Operands of = cannot be literal NULL

PostgreSQL and SQL Server return 0 rows in silence. BigQuery stops you sooner.

THE FIX

IS NULL finds the empty value

Nothing equals “no value”. **IS NULL** does the test.

```
SELECT customer_id, customer_name, city  
FROM D03.customers  
WHERE city IS NULL;
```

→ 1 row: C-455

Which customers are retail?

Returns C-204 only. Three customers are retail, spelled three ways.

```
SELECT customer_id, customer_name, segment  
FROM D03.customers  
WHERE segment = 'retail';
```

→ 1 row

Text comparison is exact. D04 fixes it with TRIM and LOWER.

Filter rows, and the order a query runs in

Three filters together, then one query that fails on purpose.

Suggested time • 9 minutes

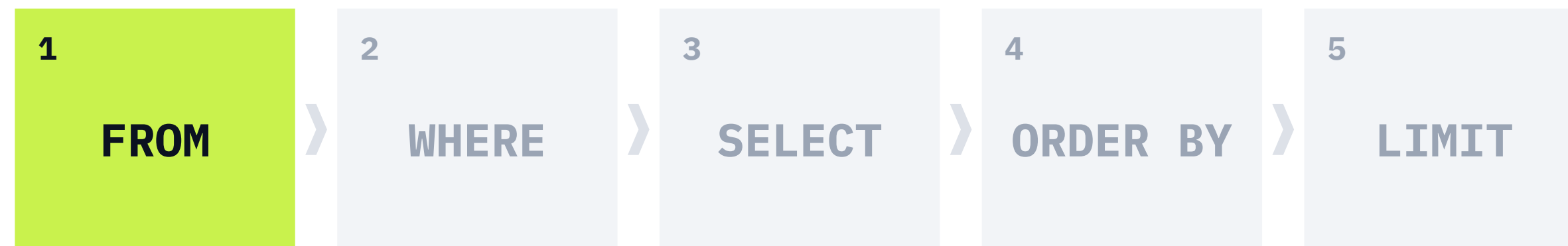
[labs/D03/05-Evr-Filter-Rows/README.md](https://github.com/DataCamp/labs/blob/main/D03/05-Evr-Filter-Rows/README.md)

The order a query runs in

YOU WRITE

SELECT FROM WHERE ORDER BY LIMIT

BIGQUERY RUNS



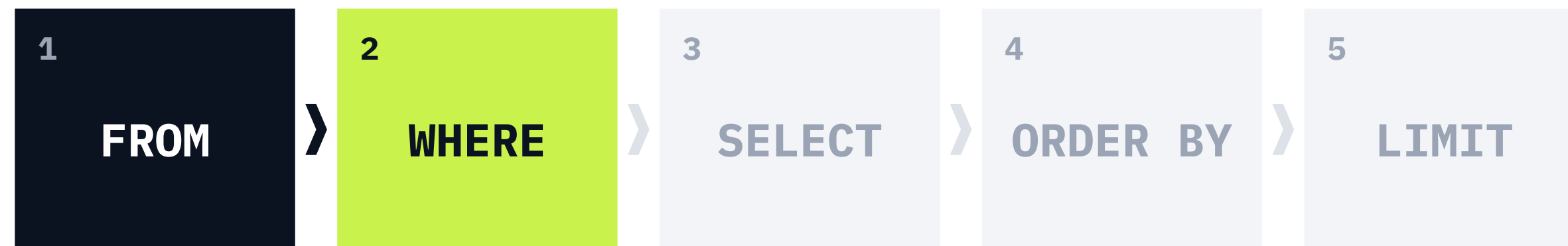
FROM runs first: which table.

The order a query runs in

YOU WRITE

SELECT FROM WHERE ORDER BY LIMIT

BIGQUERY RUNS



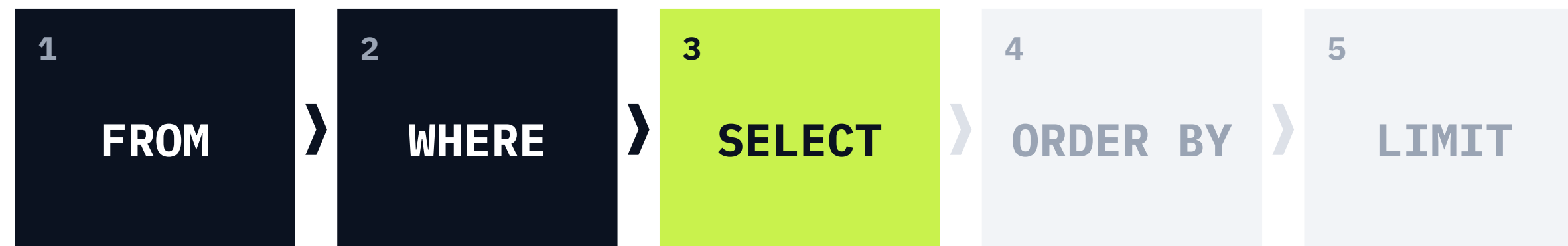
WHERE runs next: which rows. The name `order_number` does not exist yet.

The order a query runs in

YOU WRITE

SELECT FROM WHERE ORDER BY LIMIT

BIGQUERY RUNS



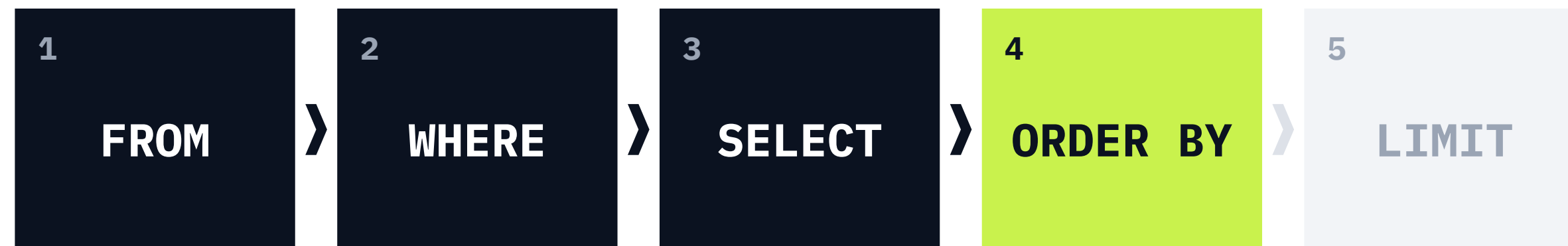
SELECT runs third: which columns, and their new names from AS.

The order a query runs in

YOU WRITE

SELECT FROM WHERE ORDER BY LIMIT

BIGQUERY RUNS



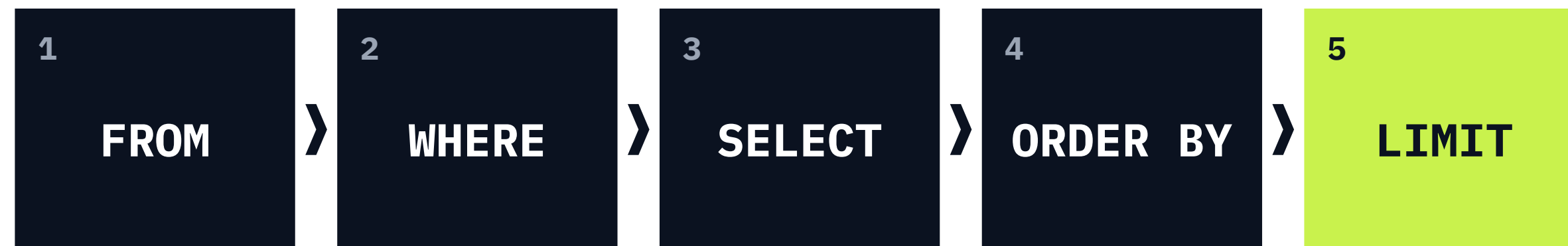
ORDER BY runs after SELECT, so sorting by order_number works.

The order a query runs in

YOU WRITE

SELECT FROM WHERE ORDER BY LIMIT

BIGQUERY RUNS



LIMIT runs last: how many rows of the sorted result to keep.

Practice, more conditions, and the lab

Four questions on your own, then AND, OR and brackets.

STUDENT DO

Activity: First questions

In this activity, you will answer four questions from the store manager using the D03 tables.

Suggested time

10 minutes

[labs/D03/06-Stu-First-Questions/README.md](https://github.com/DataCamp/labs/D03/06-Stu-First-Questions/README.md)

Activity: First questions

INSTRUCTIONS

1. Which orders were cancelled?
2. Which lines sold for more than 15 dinars a unit?
3. Which customers have no city?
4. Which orders has C-118 placed, newest first?

HINT

= NULL stops with an error. You met the fix in 4.3.

BONUS

Find all three retail customers without typing a spelling of retail.

Time's up. Let's review.

First questions

Solved: `labs/D03/06-Stu-First-Questions/Solved/first_questions.sql`

Combine conditions

AND, OR and brackets, then four shortcuts.

Suggested time • 10 minutes

[labs/D03/07-Ins-Combine-Conditions/README.md](https://github.com/DataCamp/labs/D03/07-Ins-Combine-Conditions/README.md)

Completed orders from C-118 or C-204

Predict 3 rows. It returns 4, including cancelled order 1002.

```
SELECT order_id, customer_id, status  
FROM D03.orders
```

```
WHERE customer_id = 'C-118'
```

```
OR customer_id = 'C-204'
```

```
AND status = 'completed';
```

→ 4 rows

Brackets say what you mean

AND is worked out before **OR**. Brackets change the order.

```
SELECT order_id, customer_id, status
FROM D03.orders
WHERE (customer_id = 'C-118'
      OR customer_id = 'C-204')
      AND status = 'completed';
```

→ 3 rows: 1001, 1003, 1005

IN, BETWEEN, LIKE and DISTINCT

LIKE is case-sensitive: **'%Shoe%'** finds nothing, because the product is Trail shoe.

Shortcut	Example on D03	Rows
IN	segment IN ('Retail', 'retail', 'RETAIL ')	3
BETWEEN	order_date BETWEEN '2026-08-01' AND '2026-08-11'	3
LIKE	customer_name LIKE 'Omar%'	2
DISTINCT	SELECT DISTINCT customer_id FROM D03.orders	3

Launch the lab

After the break, you load life expectancy data from Our World in Data into your own project.

Suggested time • 5 minutes

[labs/D03/08-Lab-Life-Expectancy/README.md](https://github.com/DataCamp/labs/blob/main/D03/08-Lab-Life-Expectancy/README.md)

Break

 Back at 7:00 PM

The TAs run the lab from 7:00. Ask in your group channel, with a screenshot.

Lab: life expectancy

TA-led. Our World in Data, one row per place per year.

Lab: Life expectancy

In this lab, you will load a real public dataset and answer three questions. In one, the obvious query misleads.

Suggested time

45 minutes

[labs/D03/08-Lab-Life-Expectancy/README.md](#)

Lab: Life expectancy

CHECKPOINTS

- C1 Upload `owid.life_expectancy_data`: 19,314 rows. Check that one row is one place in one year.
- C2 Jordan since 1950, and the ten highest in 2023.
- C3 Why does 2023 have 261 rows? Keep countries and territories only.

HINT

code `LIKE 'OWID%'` finds Our World in Data's own regions.

STRETCH

Bytes estimates on `thelook_ecommerce.orders`. Read them; do not run.

Time's up. Let's review.

Life expectancy

Solved: `labs/D03/08-Lab-Life-Expectancy/Solved/life_expectancy.sql`

CLOSE

What you can do now

- Say what one row is, first
- Upload a CSV and check its schema
- Select, sort, limit and filter rows
- Use IS NULL, never = NULL

NEXT

Save today's .sql file. D04: Nakheel Retail and GROUP BY.